

MaxAir Technical – Xiaomi Mijia Temperature/Humidity Sensors

Introduction

The purpose of this guide is to describe the steps required to integrate Xiaomi Mijia Temperature/Humidity sensors in to MaxAir.



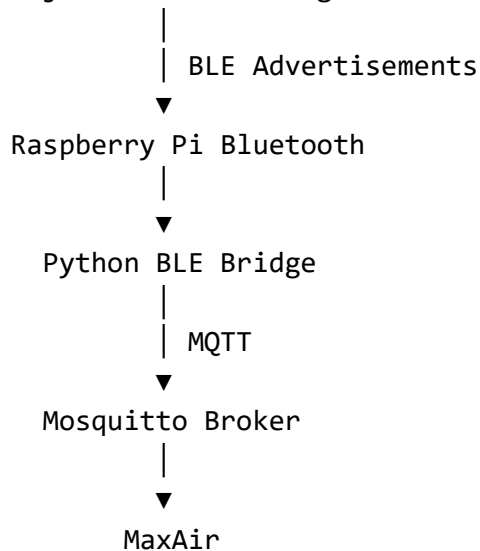
These are low cost battery powered Bluetooth compatible devices, providing both temperature and humidity readings. They are designed to work with the 'Xiaomi Home' App, but can be programmed with custom firmware to allow access via Bluetooth. For details of how to install this custom firmware see https://github.com/pv vx/ATC_MiThermometer

For the purpose of this guide, it is assumed that the custom firmware has been installed.

Integration with MaxAir

The integration will utilize the custom 'PVVX' firmware running on the device to connect to the Raspberry Pi (or some other SBC) using its Bluetooth interface. A python script using the 'bleak' python library, will capture the sensor data and forward to a Mosquitto broker as a published MQTT message, finally the MaxAir 'gateway.py' script will interpret the incoming MQTT message.

Xiaomi Mijia Sensor running PVVX firmware



Python 'bleak' Library

New builds of MaxAir using the script '/var/www/prerequisites/install.sh' will have the 'bleak' python library installed automatically. For existing builds, execute the following from the command line:

```
sudo pip3 install bleak --break-system-packages
```

Issues with the Raspberry Pi Bluetooth Interface

It would appear that for Desktop versions of the Raspberry Pi Operating System, the Bluetooth interface is enable by default and no further intervention is required.

For the 'Lite' version of Raspberry Pi Operating System, Bluetooth may not be enabled by default.

Symptom – when executing 'bluetoothctl power on', the returned response is

'Failed to set power on: org.bluez.Error.Failed'.

A solution which has been found to work is to execute the following from the command line:

```
sudo rfkill unblock bluetooth
sudo systemctl stop bluetooth
sudo systemctl start bluetooth
```

To verify that Bluetooth is operational, execute the following from the command line:

```
bluetoothctl
power on
scan on
```

Determining the Address for each Xiaomi Mijia Sensor

When configuring the devices within Maxair, the unique ID for each sensor must be obtained, this ID is based on the MAC address for the particular device and can be determined by executing the three commands shown directly above. A typical repose is:

```
maxair@maxair:~ $ bluetoothctl
Agent registered
hci0 new_settings: powered bondable ssp br/edr le secure-conn
[CHG] Controller D8:3A:DD:93:36:05 Pairable: yes
[bluetoothctl]> power on
Changing power on succeeded
[bluetoothctl]> scan on
SetDiscoveryFilter success
Discovery started
[CHG] Controller D8:3A:DD:93:36:05 Discovering: yes
[NEW] Device 47:9E:55:08:C5:89 47-9E-55-08-C5-89
[NEW] Device 46:92:BC:10:A1:70 46-92-BC-10-A1-70
[NEW] Device 46:5C:BF:F6:6A:CC 46-5C-BF-F6-6A-CC
[NEW] Device 4E:3C:BF:88:01:BD 4E-3C-BF-88-01-BD
[NEW] Device 15:25:1F:3C:B1:CB 15-25-1F-3C-B1-CB
[NEW] Device 55:C2:24:FE:FA:42 55-C2-24-FE-FA-42
[NEW] Device A4:C1:38:6A:09:E9 ATC_6A09E9
[NEW] Device A4:C1:38:42:7A:1E ATC_427A1E
[NEW] Device F0:D5:BF:14:33:FD TERRI
[CHG] Device 4E:3C:BF:88:01:BD RSSI: 0xffffffa0 (-96)
[CHG] Device A4:C1:38:6A:09:E9 ServiceData.0000181a-0000-1000-8000-00805f9b34fb:
e9 09 6a 38 c1 a4 e4 07 2d 17 4e 0b 63 65 0e ..j8....-.N.ce.
[CHG] Device A4:C1:38:42:7A:1E ServiceData.0000181a-0000-1000-8000-00805f9b34fb:
1e 7a 42 38 c1 a4 d1 08 32 14 ac 0b 64 87 04 .zB8....2....d..
[bluetoothctl]>
quit
```

The information we are interest in is the two lines indicated, the unique ID is formed from the last part of the data starting at 'ATC' then with the space removed , eg for the first device: ATC6a09e9 and for the second device: ATC427a1e.

MQTT Device Configuration

A connection to a Mosquitto broker must be configured using Settings/System Configuration/MQTT. The Xiaomi Mijia Sensors are added using Settings/Device Configuration/MQTT Devices, an example is shown below, the 'JSON Attribute' uses the ID information obtained above:

MQTT Devices 

List of MQTT Devices Attached to Node Types.

Node	Child ID	Child Name	MQTT Topic	ON Payload	OFF Payload	JSON Attribute	Notice Interval	Min Level	
31 - MQTT Sensor	1	Bedroom Temp	tele/SENSOR			ATC427a1e.Temperature	120	75	 
31 - MQTT Sensor	2	Bedroom Humid	tele/SENSOR			ATC427a1e.Humidity	0	0	 
31 - MQTT Sensor	3	Lounge Temp	tele/SENSOR			ATC6a09e9.Temperature	120	75	 
31 - MQTT Sensor	4	Lounge Humid	tele/SENSOR			ATC6a09e9.Humidity	0	0	 

Close
Add MQTT Device

For further information on MaxAir MQTT configuration, please see the document – 'Setup Guide MQTT Devices'.

Enabling the Python BLE Bridge Script

Enable using Settings System Configuration/Jobs, click to check the enable box for job 'check_pvvx_bridge' and then click on 'Apply'.

Schedule Jobs 

system_c	/var/www/cron/system_c.php	☒	☐	300	
weather_update	/var/www/cron/weather_update.php	☒	☐	1800	
nmcli_reboot_wifi	nmcli_reboot_wifi.sh	☒	☐	120	
check_ds18b20	/var/www/cron/check_ds18b20.php	☐	☐	60	
check_ebus	/var/www/cron/check_ebus.php	☐	☐	60	
check_ems	/var/www/cron/check_ems.php	☐	☐	60	
notice	/var/www/cron/notice.py	☐	☐	60	
sw_install	/var/www/cron/sw_install.py	☒	☐	10	
update_code	/var/www/cron/update_code.py	☒	☐	00:00	
check_gpio_switch	/var/www/cron/check_gpio_switch.php	☐	☐	60	
auto_backup	/var/www/cron/auto_backup.py	☒	☐	01:00	
ebus	/var/www/cron/check_ebus.php	☐	☐	60	
graphs_to_csv	/var/www/cron/graph_csv.py	☒	☐	00:00	
check_pvvx_bridge	/var/www/cron/check_pvvx_bridge.py	☒	☐	60	

Add Job
Apply
Close

Operating Characteristics

- Uses Raspberry Pi built-in Bluetooth
- Receives PVVX advertisements passively
- Does not connect or pair with sensors
- Publishes every 55 seconds (configured within `/var/www/cron/ pvvx_ble_mqtt.py`)
- Publishes valid JSON
- Publishes to:
 - Topic as set in MQTT Devices Configuration e.g. `tele/SENSOR`
- Reports:
 - Temperature
 - Humidity
 - Battery percentage